



Deliverable 1 of 3

High-Level Design Document

RAG-Based Customer Support Assistant with LangGraph and HITL

Project:	RAG-Based Customer Support Assistant
Prepared By:	Shravan Shidruk
Institute:	Innomatics Research Labs
Tech Stack:	Python, LangChain, LangGraph, ChromaDB, Ollama, Groq
Date:	April 2026

Table of Contents

1. System Overview

- 1.1 Problem Definition
- 1.2 Scope of the System

2. Architecture Diagram

- 2.1 Component Overview

3. Component Description

4. Data Flow

5. Technology Choices

6. Scalability Considerations

1. System Overview

1.1 Problem Definition

Traditional customer support systems rely on static FAQ databases or human agents, both of which struggle to scale and remain contextually accurate. This project addresses that gap by building a Retrieval-Augmented Generation (RAG) system that serves as a fully automated yet escalation-aware customer support assistant for NovaTech. The system ingests a PDF knowledge base, converts it into searchable vector embeddings, retrieves the most relevant context on demand, and passes it to a large language model to produce accurate, context-grounded answers.

1.2 Scope of the System

The system covers the following functional boundaries:

- Loading and preprocessing a PDF-based knowledge base (data/knowledge_base.pdf).
- Chunking the document using recursive character-level splitting (chunk size 700, overlap 150).
- Generating dense vector embeddings using the nomic-embed-text model via Ollama.
- Storing and querying embeddings in a persistent ChromaDB vector store.
- Routing queries through a LangGraph-based stateful workflow with four nodes: process, decision, respond, and escalate.
- Generating natural language answers through the Groq-hosted Llama 3.1 8B Instant model.
- Automatically escalating specific queries (refunds, payment issues, low-confidence answers) to a Human-in-the-Loop (HITL) module.
- Providing a CLI-based chat interface for user interaction.

The system is intentionally scoped to a single PDF knowledge base and a CLI interface. Web deployment, multi-document ingestion, and persistent conversation memory are noted as future enhancements.

2. Architecture Diagram

The diagram below illustrates the complete system architecture. The flow moves from user input through the document ingestion pipeline, into the vector store, and onward through the LangGraph workflow to produce a final response or trigger a human escalation.

Layer	Component	Technology Used
User Interface	CLI Chat Loop (main.py)	Python input() / stdout
Ingestion Pipeline	PDF Loader -> Chunker -> Embedder	PyPDFLoader, RecursiveCharacterTextSplitter, Ollama nomic-embed-text
Vector Database	ChromaDB Persistent Store	langchain-chroma, chroma.sqlite3
Retrieval Layer	MMR Retriever (k=3, fetch_k=6)	Chroma.as_retriever with MMR search
LLM Processing	Context-aware prompt + Groq LLM	ChatGroq - Llama 3.1 8B Instant
Workflow Engine	StateGraph with 4 nodes	LangGraph StateGraph
Routing Logic	Conditional edge router	router.py - keyword + confidence check
HITL Module	Human escalation handler	escalation.py - console input prompt

Table 1: System Architecture Layer Map

2.1 Component Overview

The architecture is organized into three logical tiers. The ingestion tier handles all offline processing: loading the PDF, cleaning and splitting the text, generating embeddings, and persisting them to ChromaDB. The query tier handles online processing: receiving a user query, embedding it, retrieving the top-k relevant chunks using Maximal Marginal Relevance (MMR), and constructing a prompt. The orchestration tier then manages the decision flow via LangGraph, routing to either the respond node for direct answers or the escalate node for human intervention.

3. Component Description

Each component in the system has a well-defined responsibility, as described below.

Component	Module / File	Responsibility
Document Loader	<code>ingestion/loader.py</code>	Uses PyPDFLoader from langchain_community to load the PDF and return a list of LangChain Document objects, one per page.
Chunker	<code>ingestion/chunker.py</code>	Cleans raw page text (normalizing whitespace and removing pipe characters), then applies RecursiveCharacterTextSplitter with chunk_size=700 and chunk_overlap=150, using paragraph, line, sentence, and word-level separators.
Embedding Model	<code>ingestion/embedder.py</code>	Returns an OllamaEmbeddings instance configured to use nomic-embed-text served locally at http://localhost:11434.
Vector Store	<code>retrieval/retriever.py</code>	Stores document chunk embeddings in a persistent ChromaDB instance at the vectorstore/ directory using Chroma.from_documents().
Retriever	<code>retrieval/retriever.py</code>	Loads the persistent ChromaDB store and exposes an MMR-based retriever (k=3, fetch_k=6). The retrieve_docs() function additionally deduplicates returned chunks.
LLM	<code>llm/llm.py</code>	Instantiates ChatGroq with the llama-3.1-8b-instant model. The API key is loaded from the environment via python-dotenv.
Graph Workflow Engine	<code>graph/workflow.py</code>	Defines a LangGraph StateGraph with four nodes (process, decision, respond, escalate) and conditional routing. The GraphState TypedDict carries query, answer, retriever, and llm across nodes.

Routing Layer	<code>routing/router.py</code>	The <code>route_decision()</code> function checks for escalation keywords (refund, payment issue) in the query, and checks for low-confidence indicators (answer shorter than 30 characters or containing 'not found') in the answer.
HITL Module	<code>hitl/escalation.py</code>	Prints the escalated query to the console and prompts a human operator to type a manual response, which is returned to the graph as the final answer.

Table 2: Component Description

4. Data Flow

4.1 Offline Ingestion Pipeline

This pipeline runs once (or whenever the knowledge base changes) to populate the vector store. It does not execute on every user query.

- Step 1 - Load PDF: PyPDFLoader reads data/knowledge_base.pdf and produces one Document per page, each carrying the raw page text and page metadata.
- Step 2 - Clean and Chunk: The chunker normalizes whitespace, removes noise characters, then splits the text recursively into chunks of at most 700 characters with 150-character overlap to preserve sentence context across boundaries.
- Step 3 - Embed: OllamaEmbeddings calls the locally running nomic-embed-text model to convert each chunk into a dense vector.
- Step 4 - Store: Chroma.from_documents() writes the vectors and associated text into a persistent SQLite-backed store at vectorstore/chroma.sqlite3.

4.2 Online Query Lifecycle

This pipeline executes on every user query and is orchestrated by the LangGraph workflow.

- Step 1 - User Input: The CLI loop in main.py reads a query string from stdin.
- Step 2 - Graph Entry (process node): The query is passed to the retriever, which embeds it and performs MMR search against ChromaDB, returning 3 diverse, deduplicated chunks.
- Step 3 - Prompt Construction: The retrieved chunks are concatenated into a context string and injected into a structured prompt that instructs the LLM to answer only from the provided context.
- Step 4 - LLM Invocation: ChatGroq sends the prompt to Llama 3.1 8B Instant and returns a response.content string.
- Step 5 - Decision Node: route_decision() inspects the query and answer for escalation triggers. It returns either 'respond' or 'escalate'.
- Step 6a - Respond Node: If routed to respond, the state is returned as-is and the answer is printed to the CLI.
- Step 6b - Escalate Node: If routed to escalate, human_intervention() prints the query and prompts a human operator, whose typed response becomes the final answer.

5. Technology Choices

Technology	Version	Reason for Choice
ChromaDB	1.5.8	Lightweight, file-based vector database requiring no external server. Persistent SQLite backend makes local development straightforward. Native LangChain integration via langchain-chroma.
LangGraph	1.1.9	Provides first-class support for stateful, multi-node graph workflows. TypedDict-based state management ensures type safety across nodes. Built-in support for conditional edges aligns directly with the routing requirement.
Groq (Llama 3.1 8B Instant)	API	Groq's inference API delivers very low latency without requiring local GPU hardware. Llama 3.1 8B Instant is capable for instruction-following tasks at minimal cost.
Ollama (nomic-embed-text)	Local	Running embeddings locally avoids API costs and latency for the embedding step. nomic-embed-text is a high-quality open embedding model optimized for retrieval tasks.
LangChain	1.2.15	Provides the document loading, text splitting, and retriever abstractions that standardize the pipeline and reduce boilerplate code.
sentence-transformers	5.4.1	Available as a fallback embedding option and used by Ollama under the hood for model serving.

Table 3: Technology Stack with Justifications

6. Scalability Considerations

6.1 Handling Large Documents

The current ingestion pipeline processes one PDF into ChromaDB at a time. To handle large documents, the `chunk_size` and `chunk_overlap` parameters in `RecursiveCharacterTextSplitter` can be tuned. For very large PDFs, batch insertion into ChromaDB using `add_documents()` in configurable batch sizes prevents memory overflow. ChromaDB supports collections, which would allow documents to be stored in named namespaces and queried selectively.

6.2 Increasing Query Load

The current CLI implementation is single-threaded and sequential. Moving to a FastAPI or async framework would allow concurrent query handling. The ChromaDB vector store is file-based and supports read-heavy concurrent access. For high throughput, the vector store can be migrated to a hosted solution such as Pinecone or Weaviate, which provide horizontal scaling. The Groq API itself is designed for high-concurrency inference.

6.3 Latency Concerns

The primary latency source in the current architecture is the Ollama embedding call, which runs on CPU. Pre-computing embeddings during ingestion (already done) means query-time embedding is limited to the single query vector. The MMR retrieval with `fetch_k=6` adds a small overhead over pure cosine similarity but improves result diversity. Caching frequently asked queries with their retrieved contexts would further reduce latency for repeated questions.

End of Deliverable 1 - High-Level Design